Accuracy, Stability, and Repeated-Run Reliability of Large Language Models on Deterministic Programming Tasks

Yongxi Zhou
Northeastern University
Massachusetts, USA
zhou.yongx@northeastern.edu

Abstract

Run-level pass rate overstates retry-free coverage by up to 17.8 percentage points—and the gap is largest precisely for mid-performing systems. We investigate this accuracy–stability relationship in large language model (LLM) evaluation for deterministic text-conditioned generation, using programming tasks as a concrete testbed. Standard code-generation benchmarks emphasize single-run accuracy or eventual success under repeated sampling, but many deployment settings also require *stability*: consistent outcomes across repeated invocations under the same task description. We present a repeated-run evaluation protocol with metrics for run-level accuracy, retry-free coverage, and per-problem variability. On a recency-based benchmark of 100 LeetCode-style problems, we evaluate 16 models from five provider families under two prompt templates with five repeated runs per problem, yielding 16,000 evaluation instances. Although run-level pass rate and perfect stability rate are strongly correlated ($r = 0.985$), pass rate consistently exceeds retry-free coverage—a gap that reaches 17.8 percentage points and reverses model rankings even among closely matched systems. Prompt effects are model-dependent rather than uniformly beneficial. These results suggest that repeated-run stability analysis is a necessary complement to conventional accuracy reporting for deterministic text-conditioned generation tasks.

1 Introduction

Large language models are increasingly used for text-conditioned tasks with deterministic specifications: given a fixed textual input and a fixed verifier, correctness is binary. Program synthesis and code assistance are especially important instances of this broader setting, because they pair natural-language problem statements with executable test suites. Conventional benchmarks typically report single-run performance, or summarize repeated sampling through eventual-success metrics such as pass@k (Chen et al., 2021; Austin et al., 2021). Recent work has shown that repeated sampling can be scaled as a form of inference-time compute (Brown et al., 2024), yet the *stability* of outcomes across such repeated invocations—whether a model consistently succeeds or fails on the same task—has received less systematic attention. In practice, repeated calls under the same task description can yield different outputs and different pass/fail outcomes, even when the downstream workflow expects reproducible behavior (Atil et al., 2024; Alvarado Gonzalez et al., 2025).

This paper studies that repeated-invocation variability directly. Rather than treating repeated sampling only as an opportunity to recover one success among many attempts, we investigate the relationship between correctness and stability under fixed conditions. Our focus is therefore complementary to pass@k (Chen et al., 2021): whereas pass@k asks whether a model can produce *at least one* correct solution in k attempts—a capability measure—we ask how reliably it succeeds on *every* invocation, a deployment reliability measure. These two questions target different downstream concerns and can yield divergent rankings, particularly for mid-accuracy systems.

The paper is organized around three empirical questions:

1. How large is the gap between run-level success and retry-free problem coverage under repeated evaluation?

2. Which aggregate summaries best expose problem-level variability that single-run reporting hides?

3. As an exploratory analysis: do prompt choices or reasoning-specialized architectures systematically shift repeated-run stability?

We focus on deterministic tasks because they represent a broader and practically important class of NLP system behavior: the model is conditioned on text, but the output is consumed by a verifier that expects reproducible correctness. Code-generation systems are a particularly stringent instance of this pattern, alongside tasks such as text-to-SQL (Gao et al., 2024), structured semantic parsing, tool-call argument generation, and formal text transformations. In such settings, the accuracy–stability gap has direct practical consequences: a model with 60% run-level pass rate but only 47% perfect stability solves many individual attempts, yet still leaves a substantial fraction of tasks in a retry-required regime. Our aim is therefore not merely to rank models by mean performance, but to characterize how much of their observed accuracy translates into reliable repeated-use behavior.

2 Related Work

Code-generation evaluation is commonly framed around functional correctness and pass@k under automated testing (Chen et al., 2021; Austin et al., 2021). Foundational benchmarks emphasize whether a model eventually produces at least one correct program under repeated sampling (Chen et al., 2021; Austin et al., 2021; Hendrycks et al., 2021), while more recent efforts address contamination and temporal validity (Jain et al., 2025; Jimenez et al., 2024). Prompting studies—including zero-shot reasoning and self-consistency—similarly use multiple samples as a mechanism for improving performance rather than as an object of measurement (Kojima et al., 2022; Wang et al., 2023; Aggarwal et al., 2023). In parallel, work on LLM output non-determinism shows that performance varies substantially across repeated runs under identical conditions (Atil et al., 2024; Alvarado Gonzalez et al., 2025), and prompt sensitivity research demonstrates that small phrasing changes can shift accuracy (Zhuo et al., 2024).

PSR is mathematically equivalent to pass@k at $k = R$ with an all-successes threshold; the distinction is interpretive. Pass@k estimates $P(\text{at least one success in } k \text{ draws})$—a *capability* measure—while PSR estimates $P(\text{all } k \text{ draws succeed})$—a *deployment reliability* measure. These quantities diverge most sharply for mid-accuracy systems. The contribution lies in treating PSR and AV as first-class evaluation targets for deterministic code tasks and using them to characterize the accuracy–stability gap directly.

Recent work emphasizes that code evaluation is sensitive to benchmark construction and reporting choices (Liu et al., 2023; Du et al., 2024). Our framing shifts the emphasis from benchmark validity to repeated-invocation reliability under a fixed judge. Work on calibration (Guo et al., 2017) and performance–efficiency trade-offs (Zhang et al., 2026) further motivates looking beyond average accuracy when consistency matters. This perspective is deliberately narrower than end-to-end agent evaluation: it isolates repeated-run behavior for single-turn coding, while broader orchestration, tool use, and runtime dependencies require separate analysis.

3 Experimental Setup

3.1 Problem Dataset

We evaluate on a curated dataset of deterministic programming problems, denoted $\mathcal{D} = \{(x_i, T_i)\}_{i=1}^{N}$, where x_i is the natural-language specification (including I/O format and constraints) and T_i is a deterministic test harness. Each problem is graded by executing the model-produced program against T_i, producing a binary correctness indicator. These tasks are instantiated as LeetCode problems with platform-provided acceptance criteria, and all generations are evaluated as Python solutions against Python-based submission templates.

Problem selection. To reduce the risk of benchmark contamination from model training data, we select the $N = 100$ most recently published problems available on the platform at collection time, sampling the newest problems first. Problems requiring paid-only access are excluded. No further manual curation is applied; the final benchmark is entirely determined by recency rank. This construction does not guarantee that evaluated models have never encountered related problem statements during training, but it substantially reduces the probability of direct memorization compared to historically popular problems. We note, however, that recency alone does not guarantee a contamination-free evaluation: recently published contest problems are often discussed publicly online—including on forums, social media, and editorial sites—within days of release. Post-release discussion means that problem statements, constraints, and accepted solutions may appear in model training corpora even for recently added

problems, and we do not perform contamination-detection analysis (e.g., via prompt perturbation, n-gram overlap, or membership inference (Deng et al., 2024)). This limitation is discussed further in the Limitations section.

The dataset spans three difficulty tiers: 20 Easy, 50 Medium, and 30 Hard problems, covering a broad range of algorithmic topics including Array, Dynamic Programming, String, Graph, and Simulation. All problems satisfy the following properties:

- **Deterministic grading:** each T_i yields a deterministic accept/reject outcome.
- **Self-contained evaluation:** no network access or external dependencies are required at runtime.
- **Fixed snapshot:** all models are evaluated on the identical problem set to ensure comparability across runs.
- **No personal data:** problems consist of algorithmic specifications only and contain no personally identifying information.

In the evaluation framework, this benchmark is stored as a fixed local dataset snapshot rather than fetched online at run time. This design ensures that all model families are evaluated against the same problem set and ordering, and reduces the chance that later platform changes silently alter the benchmark during the experimental campaign.

3.2 Models

We evaluate 16 models across five provider families, covering a range of model scales, training objectives, and architectural choices, including both standard and reasoning-specialized variants. The evaluated models are listed in Table 1.

Table 1: Models evaluated in this study, grouped by provider family. † denotes reasoning-specialized models. Display names in this table are editorial labels derived from API identifier strings; they are not official product version names. Exact API identifiers and access dates are listed in Appendix B. Google models marked "(preview)" were accessed via preview-tier API endpoints.

Family	Model	Type
OpenAI	GPT-4.1	flagship
	GPT-4.1-mini	lightweight
	o4-mini	reasoning†
Anthropic	Claude Sonnet 4.5	flagship
	Claude Haiku 4.5	lightweight
Google	Gemini 3.1 Pro (preview)	flagship
	Gemini 3 Flash (preview)	efficient
	Gemini 3.1 Flash-Lite (preview)	lightweight
	Gemini 2.5 Flash	efficient
	Gemini 2.5 Flash+Think	reasoning†
Qwen	QwQ-Plus	reasoning†
	Qwen-Plus	flagship
	Qwen-Max	large
	Qwen-Turbo	lightweight
DeepSeek	DeepSeek-R1	reasoning†
	DeepSeek-V3.2	flagship

3.3 Evaluation Framework

For a model m and a fixed evaluation configuration c (prompt template, decoding parameters, toolchain), we run each problem x_i multiple times. Each run produces a candidate program which is executed against T_i.

Let $Y_{i,r}^{(m,c)} \in \{0,1\}$ denote whether run $r \in \{1,\ldots,R\}$ passes all tests for problem i. We treat compilation errors, runtime errors, timeouts, and wrong answers as failures (0).

The evaluation pipeline separates generation from execution. For large-scale experiments, model outputs are first generated through provider APIs, cached locally, normalized into a shared format, and only then submitted to the deterministic judge. This design makes experiments resumable across providers and reduces the risk that transient provider-side failures are conflated with downstream execution outcomes.

3.4 Repeated-Run Methodology

We adopt a repeated-run design to estimate both mean performance and variability. For each (m, c) pair we:

1. Sample R independent generations per problem under configuration c.
2. Evaluate each generation deterministically against T_i.
3. Aggregate outcomes into run-level and problem-level metrics with uncertainty estimates.

The primary controlled intervention is the prompt template. We compare two prompt variants while holding the dataset, model, decoding parameters, and execution environment fixed. All models are evaluated with temperature 0.3, top-$p = 0.9$, and $R = 5$ repeated runs per problem per prompt configuration. The default maximum output budget

is 4,096 tokens; reasoning-specialized models (o4-mini, Gemini 2.5 Flash+Think) use larger budgets to accommodate extended chain-of-thought output (see Appendix B for per-model details). The o-series API (o4-mini) does not support explicit temperature or top-p settings; decoding parameters are omitted for that model (implications discussed in the Limitations section). We use a non-zero temperature because the goal of the study is to characterize repeated-run variability under realistic stochastic decoding rather than to collapse outputs toward near-deterministic behavior at $T = 0$. The two prompt templates—DETAILED and MINIMAL—are held fixed across all models to enable a controlled comparison of prompt sensitivity.

Prompt templates. Both prompts instruct the model to return plain Python source code only and to follow the provided LeetCode method signature exactly. The DETAILED prompt uses a longer structured template emphasizing careful constraint reading, edge-case coverage, submission readiness, and accepted-style performance. The MINIMAL prompt conveys the same task more compactly, with lighter instruction scaffolding. Because both templates share the same output constraints and code template, the prompt comparison is a controlled low-contrast comparison that isolates instruction density rather than changing the task itself. Accordingly, the study does not test broad prompt-engineering strategies; it evaluates a narrow prompt variation under otherwise fixed conditions.

Normalization and execution. Raw generations from different providers are normalized into a common local representation before evaluation. Code is extracted from model responses, basic syntax validity is checked, and each candidate is then submitted to the same deterministic judge. This normalization step is important because provider APIs differ in output packaging, token accounting, and batch interfaces even when the downstream task is identical.

3.5 Metrics Definition

Run-Level Pass Rate (RLPR).

$$\mathrm{RLPR}(m,c) = \frac{1}{NR}\sum_{i=1}^{N}\sum_{r=1}^{R} Y_{i,r}^{(m,c)}.$$

RLPR measures the probability that a *random invocation* succeeds.

Perfect Stability Rate (PSR).

$$\mathrm{PSR}(m,c) = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}\left[\sum_{r=1}^{R} Y_{i,r}^{(m,c)} = R\right].$$

PSR quantifies the fraction of tasks for which the system is reliably correct without retries.

Average Variance (AV). $\mathrm{AV}(m,c) = \frac{1}{N}\sum_{i=1}^{N}\frac{R}{R-1}\hat{p}_i(1-\hat{p}_i)$ with $\hat{p}_i = \frac{1}{R}\sum_r Y_{i,r}$ captures average instability across tasks.

Inference. We report 95% Wilson score intervals for RLPR and PSR, and nonparametric bootstrap CIs for AV. Since both prompt templates are evaluated on the same problems, we compare PSR across prompt conditions using exact McNemar tests on the binary indicator of whether all $R = 5$ runs pass. Reported statistics are descriptive summaries of the observed protocol rather than universal claims; prompt comparisons are specific to the two templates used here.

4 Results

4.1 Overall Accuracy and Stability

Table 2 reports RLPR, PSR, and AV for all 32 model/prompt configurations. The top performers under RLPR are o4-mini (86.2% D / 85.6% M), Gemini 3.1 Pro (86.2% D / 85.8% M), and DeepSeek-R1 (84.2% D / 84.4% M). More broadly, the table shows that high run-level performance and high retry-free coverage often co-occur, but not in a one-to-one way: models with similar RLPR can still differ materially in PSR and AV.

Figure 1 plots RLPR against PSR for all 32 configurations. All points fall below the diagonal, confirming that RLPR consistently overstates production reliability. Across all configurations, RLPR and PSR are strongly correlated ($r = 0.985$, $p < 0.001$), which is directionally expected because both metrics are derived from the same underlying per-problem success structure. The informative result is therefore not the existence of a high correlation itself, but the persistent and non-uniform gap between run-level success and retry-free coverage.

The gap $\Delta = \mathrm{RLPR} - \mathrm{PSR}$ is non-uniform and reaches its maximum in the mid-accuracy tier. Gemini 2.5 Flash has the largest gap (17.8% under DETAILED), followed by Gemini 3.1 Flash-Lite (16.2% D) and Gemini 2.5 Flash+Think (17.4% M). At the low end, Qwen-Turbo has a gap of only

Table 2: Main results across all 16 models and two prompt configurations (100 problems, $R = 5$ runs, 16,000 total evaluation instances). RLPR and PSR are in %; AV is shown $\times 10^{-2}$ (i.e., multiply by 0.01 for the raw value). All entries are point estimates; 95% CIs (Wilson score for RLPR/PSR; bootstrap for AV) available from authors on request. † reasoning-specialized model. $*$ output budget differs from the 4,096-token default: Gemini 2.5 Flash+Think used 24,576 tokens (8,192 thinking); o4-mini used 16,384 tokens. ‡ o4-mini decoding is not user-controllable; its PSR/AV may reflect reduced stochasticity rather than task reliability—see the Limitations section.

Family	Model	RLPR (%)		PSR (%)		AV ($\times 10^{-2}$)	
		D	M	D	M	D	M
OpenAI	GPT-4.1	59.6	59.2	47.5	49.0	4.36	4.08
	GPT-4.1-mini	58.4	59.2	50.0	47.0	3.28	4.48
	o4-mini†‡	86.2	85.6	75.0	69.0	3.36	5.20
Anthropic	Claude Sonnet 4.5	52.8	51.4	43.0	47.0	3.36	1.68
	Claude Haiku 4.5	41.2	40.8	35.0	34.0	2.40	2.00
Google	Gemini 3.1 Pro (preview)	**86.2**	85.8	**75.0**	74.0	3.36	3.92
	Gemini 3 Flash (preview)	80.8	**83.2**	67.0	70.0	5.28	4.40
	Gemini 3.1 Flash-Lite (preview)	69.2	63.6	53.0	50.0	5.28	5.44
	Gemini 2.5 Flash*	53.8	48.2	36.0	33.0	7.04	5.84
	Gemini 2.5 Flash+Think†*	71.6	74.4	58.0	57.0	4.64	4.72
Qwen	QwQ-Plus	72.8	72.6	59.0	58.0	4.80	4.72
	Qwen-Plus	63.4	61.2	52.0	51.0	4.72	4.64
	Qwen-Max	30.6	30.4	26.0	25.0	2.24	2.24
	Qwen-Turbo	34.4	30.8	32.0	28.0	0.88	1.04
DeepSeek	DeepSeek-R1	84.2	**84.4**	74.0	**75.0**	3.76	3.20
	DeepSeek-V3.2	48.4	47.6	36.0	38.0	5.52	4.00

2.4%—not because it is uniformly reliable, but because its outcomes are highly polarized: many problems are either always failed or always passed, leaving less room for intermediate trial-to-trial variability. This pattern is reflected in AV: Qwen-Turbo has the lowest AV (0.0088), while Gemini 2.5 Flash has the highest (0.0704). Together, PSR and AV help distinguish between systems that are consistently strong, consistently weak, and unstable in the middle.

While it is mathematically expected that RLPR $\geq$ PSR for any non-trivial Bernoulli process, and that the two metrics are highly correlated overall, the *non-uniformity* of Δ across models is the substantive empirical finding. This non-uniformity is itself structurally expected: Bernoulli variance is maximized at $p \approx 0.5$, so mid-performing models—whose per-problem success probabilities cluster in the intermediate range—will exhibit the largest gaps by construction. The practical contribution is that this gap is observable, model-specific, and meaningfully large (up to 17.8 percentage points), meaning that RLPR rankings alone cannot reveal how much of a model's accuracy translates into retry-free reliability.

Knowing the theoretical direction does not eliminate the need to measure the magnitude. The empirically determined quantities—how large Δ actually is (up to 17.8 pp) and which specific models fall in the mid-accuracy danger zone—cannot be derived from first principles, since per-problem success probabilities are not known a priori. The contribution is therefore best understood as empirical calibration of a theoretically expected pattern at scale.

Figure 1: RLPR vs. PSR scatter plot across all 32 configurations (16 models $\times$ 2 prompt types). Points are colored by model family and shaped by prompt type. The diagonal marks $\mathrm{PSR} = \mathrm{RLPR}$; points below indicate configurations where mean success overstates stable problem coverage.

4.2 Problem-Level Stability Analysis

Figure 2 presents a cross-model stability overview. Each row corresponds to one of the 16 models, ordered by RLPR under DETAILED prompting (highest at top). Each column corresponds to one of the 100 problems, grouped by difficulty tier (Easy, Medium, Hard) and sorted within each tier by decreasing mean pass rate across all models. Cell color encodes the empirical pass probability for that model–problem pair, averaged over $R = 5$ runs: green indicates a problem solved consistently on every run, red indicates consistent failure, and yellow indicates high trial-to-trial variability.

Figure 3 shows the distribution of per-problem pass rates aggregated across all models. Qwen-Turbo shows the most polarized profile (AV $=$ 0.0088), while Gemini 2.5 Flash has the highest AV (0.0704), indicating substantial trial-to-trial variability.

This view is useful because it exposes qualitatively different reliability profiles that similar aggregate pass rates can hide. A model with many $\hat{p}_i \approx 1$ and $\hat{p}_i \approx 0$ problems behaves differently from a model with many $\hat{p}_i \approx 0.4$ or 0.6 problems, even when their average run-level pass rate is similar.

We further note that retry-based recovery does not eliminate instability. PSR directly measures the fraction of tasks that do not require retries, while AV highlights how much of the remaining workload lies in a stochastic regime.

4.3 Difficulty Gradient and Failure Modes

Pooling outcomes by benchmark difficulty reveals a pronounced gradient: run-level pass rates are highest on Easy problems (90.6%), lower on Medium (62.2%), and substantially lower on Hard (34.0%). The dominant non-accept verdict is *Wrong Answer* ($\sim$58%), followed by *Time Limit Exceeded* ($\sim$27%) and *Runtime Error* ($\sim$11%). Most failures thus arise from incorrect algorithms rather than formatting noise, though timeouts and runtime failures increase on harder problems, indicating that instability is also tied to implementation robustness. A low PSR can therefore reflect two regimes—algorithmic inconsistency and implementation fragility—which imply different mitigation strategies.

4.4 Reasoning vs. Standard Models

Reasoning-specialized models generate an internal chain-of-thought before producing a final answer, which could plausibly improve output consistency by structuring the solution process. We test whether this architectural distinction manifests as a systematically smaller accuracy–stability gap after controlling for overall accuracy level.

Grouping. We treat four models as reasoning-specialized (marked † in Table 1): DeepSeek-R1, QwQ-Plus, o4-mini, and Gemini 2.5 Flash+Think. The remaining 12 systems are classified as standard models. All conclusions from this analysis remain exploratory given the modest group sizes.

RLPR-controlled gap analysis. Because $\Delta = \mathrm{RLPR} - \mathrm{PSR}$ is itself correlated with RLPR (higher-accuracy models tend to have larger absolute gaps, as shown in Section 4.6), a raw comparison of mean gaps between groups would be confounded by accuracy level. To address this, we fit a linear model $\Delta \sim \mathrm{RLPR}$ across all 32 configurations ($R^2 = 0.40$, $p < 0.001$) and examine the residuals: a negative residual indicates a smaller gap than expected at the model's accuracy level, i.e., greater stability relative to accuracy.

Figure 4 shows the result. Reasoning models show strongly heterogeneous behavior: DeepSeek-R1 falls well below the regression line (mean residual ≈ -4.4), suggesting markedly better stability than expected for its accuracy level; o4-mini is near

Figure 2: Cross-model stability heatmap (DETAILED prompt). Rows: 16 models sorted by RLPR descending. Columns: 100 problems grouped by difficulty tier with dashed separators, and ordered within each tier by decreasing mean pass rate. Cell color encodes empirical pass rate over $R = 5$ runs (green $= 1.0$, red $= 0.0$, yellow ≈ 0.5).

the line (residual ≈ -0.5); while QwQ-Plus (residual $\approx +1.7$) and Gemini 2.5 Flash+Think (residual $\approx +3.0$) both fall above it, showing larger-than-expected gaps. Across all eight reasoning-model configurations, the mean residual is -0.04, compared with $+0.01$ for the 24 standard-model configurations—a difference of 0.05 percentage points. A permutation test (10,000 iterations, two-sided) yields $p = 0.97$, providing no evidence that the residual difference is systematic.

Variance comparison. Average variance (AV) provides a complementary view of trial-to-trial variability. The four reasoning models have a mean AV of 4.30, compared with 3.81 for the 12 standard models—again showing no systematic advantage for reasoning-specialized architectures on this metric.

Interpretation. The four reasoning models do not exhibit systematically smaller accuracy–stability gaps or lower AV once accuracy is accounted for. DeepSeek-R1 shows suggestive evidence of above-expected stability (residual ≈ -4.4 pp), warranting verification in a larger sample. However, with only four reasoning models, the analysis is severely underpowered: the permutation test ($p = 0.97$) should be read as "insufficient evidence from a small sample" rather than "the effect is negligible." The pattern is model-specific: chain-

Figure 3: Problem-level success distribution (DETAILED prompt). The figure contains one panel per model (16 panels total), arranged in decreasing order of PSR. Each panel shows the fraction of that model's 100 problems falling in each empirical pass-probability bin $\hat{p}_i \in \{0/5, 1/5, \ldots, 5/5\}$, where $0/5$ means the model failed all 5 runs and $5/5$ means it passed all 5.

Figure 4: Accuracy–stability gap (Δ) vs. RLPR for all 32 configurations. Reasoning models (diamonds) and standard models (circles) are shown against an OLS fit. Points below the line have smaller gaps than expected at their accuracy level.

of-thought reasoning neither reliably improves nor worsens stability within this sample. One important caveat: the Gemini 2.5 Flash vs. Flash+Think comparison is confounded by output token budget (4,096 vs. 24,576 tokens), so the pair should not be interpreted as a clean ablation of thinking mode alone.

4.5 Prompt Sensitivity

Figure 5 visualizes the prompt effect as a paired scatter plot: each of the 16 models appears as a single point with its MINIMAL-prompt PSR on the x-axis and DETAILED-prompt PSR on the y-axis. Points above the diagonal indicate that detailed prompting improves stability.

Prompt effects are model-dependent: most models show small differences (within 3–5 percentage points of PSR), but Claude Sonnet 4.5 shows a reversal (higher PSR under MINIMAL), while Gemini 3 Flash shows the opposite. No single template uniformly dominates—consistent with prior findings on prompt sensitivity (Zhuo et al., 2024). Exact McNemar tests on the paired PSR indicators find no model reaching $p < 0.05$; the largest shifts are 4 points ($p = 0.219$). Across 16 models, 11 favor DETAILED and 5 favor MINIMAL (mean absolute difference $\sim$2.1 pp). This is a deliberately low-contrast comparison—both templates share identical structure, differing only in instruction density—so the null result may partly reflect insufficient contrast rather than a true absence of prompt effects. With $N = 100$, a 4-point shift has less than 20% power, meaning meaningful effects cannot be ruled out at the current sample size.

4.6 Accuracy–Stability Gap

Although RLPR and PSR are strongly correlated overall ($r = 0.985$), the gap $\Delta = \mathrm{RLPR} - \mathrm{PSR}$ is practically significant and non-uniform across models. Across all 32 configurations, Δ ranges from 2.4% (Qwen-Turbo, DETAILED) to 17.8% (Gemini 2.5 Flash, DETAILED). The gap peaks in the middle of the performance spectrum, where models succeed frequently enough to generate trial-to-trial variability but not consistently enough to eliminate it. This non-linearity means that a single RLPR ranking cannot be used to infer the magnitude of the accuracy–stability gap: two models with similar RLPR can differ substantially in how much of their accuracy converts to retry-free reliability. PSR makes this distinction explicit.

RLPR and PSR can disagree on model rankings despite the high correlation. For example, GPT-4.1 leads GPT-4.1-mini on RLPR (59.6% vs. 58.4%) but trails on PSR (47.5% vs. 50.0%)—a reversal driven by GPT-4.1's larger accuracy–stability gap (12.1 pp vs. 8.4 pp). This confirms that PSR captures deployment-relevant distinctions invisible to RLPR rankings alone.

Figure 5: Prompt sensitivity: paired scatter of PSR under DETAILED vs. MINIMAL prompting for all 16 models. Points above the diagonal indicate that detailed prompting improves stability. Points are colored by model family.

5 Conclusion

We presented a repeated-run evaluation framework for deterministic programming tasks and used it to study the relationship between correctness and stability. Across 16 models, 100 problems, and 5 repeated runs per configuration (16,000 total instances), run-level pass rate and retry-free coverage are strongly correlated, but run-level accuracy consistently exceeds perfect stability. The accuracy–stability gap is non-uniform and reaches up to 17.8 percentage points, peaking in the mid-accuracy tier. Importantly, PSR and RLPR can reverse model rankings even at high correlation ($r = 0.985$): GPT-4.1 leads GPT-4.1-mini on RLPR but trails on PSR, demonstrating that the two capture deployment-relevant distinctions invisible to single-run reporting. We view these metrics as complementary: RLPR characterizes average invocation success, while PSR and AV reveal whether that success is concentrated in consistently solved problems or dispersed across unstable outcomes. Prompt effects and the reasoning-vs-standard distinction are both heterogeneous and exploratory at the current sample size, suggesting that neither is a reliable lever for improving stability without model-specific tuning.

Future work should extend this framework in two directions: (1) replicating the repeated-run analysis on at least one non-coding deterministic NLP task (e.g., text-to-SQL) to validate generalizability, and (2) controlling output token budgets across standard and reasoning variants to cleanly isolate the effect of chain-of-thought reasoning on stability. While our evaluation isolates single-turn correctness, practical systems involve broader orchestration and runtime dependencies whose reliability and security properties require separate analysis (Jiang et al., 2026). Taken together, these findings suggest that repeated-run evaluation is a necessary complement to standard accuracy reporting for any deterministic text-conditioned generation task in reproducibility-sensitive workflows.

Limitations

Several limitations qualify our results. First, the benchmark is relatively small ($N = 100$) and restricted to recent LeetCode-style problems. This design helps reduce direct overlap with heavily circulated historical benchmarks, but it does not eliminate contamination risk and should not be interpreted as proof of benchmark novelty. In particular, recently published contest problems are frequently discussed online within days of their release, meaning that recency alone is not sufficient to guarantee that evaluated models have not been exposed to equivalent problem statements through post-publication web text. The benchmark size is also a practical tradeoff: with 16 models, 2 prompt conditions, and 5 repeated runs per problem, even a 100-problem setup yields 16,000 judged evaluation instances. This scale is large enough to expose broad correctness–stability patterns, but still limited for detecting small prompt effects, which is consistent with the non-significant prompt comparisons reported in Section 4.

Second, our repeated-run protocol is intentionally narrow: we vary prompt template and repeated sampling while holding the evaluation harness fixed. As a result, the study does not characterize the effects of broader prompt engineering, tool use, self-repair, retrieval, test-time selection, or multi-turn interaction.

Third, model access is mediated through provider APIs and platform infrastructure. Even though task grading is deterministic, we do not control all backend implementation details, model refreshes, or service-side changes that may affect outputs over time. Our conclusions should therefore be interpreted relative to documented model snapshots and access dates rather than as timeless statements about any provider family.

Fourth, our summary metrics compress distinct failure modes into a binary pass/fail outcome. This

is appropriate for end-task correctness, but it does not by itself explain whether instability arises from reasoning errors, formatting mistakes, runtime failures, or judge-specific edge cases.

Fifth, a specific methodological caveat applies to o4-mini. The o-series API does not expose temperature or top-p controls; the model's internal decoding regime is not documented and may differ substantially from the $T = 0.3$ setting used for all other models. If o4-mini's effective temperature is near zero—producing near-deterministic outputs across runs—then its measured PSR and AV reflect a qualitatively different quantity from the other systems: we would be observing the near-absence of stochasticity rather than meaningful stability under repeated stochastic sampling. This limits direct comparability of o4-mini's repeated-run metrics with those of other models, and any high PSR for o4-mini should be interpreted cautiously as a potential artifact of reduced output variance rather than as evidence of robust problem-solving reliability.

Finally, our conclusions are limited to Python code generation on deterministic coding tasks with executable tests. They should not be read as a complete account of reliability for other programming languages, for open-ended generation tasks, or for production systems that include additional orchestration and verification layers.

Ethics Statement

This paper evaluates commercial and open-access LLM APIs on deterministic programming tasks. The study does not involve human subjects, personal data collection, or annotation labor. The main ethical considerations are instead tied to platform use and deployment interpretation. First, the evaluation relies on automated submission to an external online judge. Such use should be understood as research benchmarking under the platform's operational constraints, and any public release should avoid exposing credentials, session artifacts, or procedures that would enable abusive automation. Second, the evaluated models are accessed through third-party APIs whose outputs and availability may be governed by provider-specific terms of service and may change over time. Our reported results should therefore be interpreted as measurements of accessed systems under a documented evaluation protocol, not as immutable properties of any provider's model family.

More broadly, stronger performance on coding benchmarks can increase both beneficial and harmful capabilities. Better repeated-run reliability can support legitimate software engineering workflows, but it may also lower the barrier to generating code in settings where security review is weak. For this reason, we frame our contribution as an evaluation methodology rather than as an endorsement of fully autonomous code deployment.

References

Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. 2023. Let's sample step by step: Adaptive-consistency for efficient reasoning and coding with LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12375–12396, Singapore. Association for Computational Linguistics.

Miguel Angel Alvarado Gonzalez, Michelle Bruno Hernandez, Miguel Angel Peñaloza Perez, Bruno Lopez Orozco, Jesus Tadeo Cruz Soto, and Sandra Malagon. 2025. Do repetitions matter? Strengthening reliability in LLM evaluations. *Preprint*, arXiv:2509.24086.

Berk Atil, Alexa Chittams, Liseng Fu, Ferhan Ture, Lixinyu Xu, and Breck Baldwin. 2024. LLM stability: A detailed analysis with some surprises. *Preprint*, arXiv:2408.04667.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, Charles Sutton, and Wojciech Zaremba. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. 2024. Large language monkeys: Scaling inference compute with repeated sampling. In *Advances in Neural Information Processing Systems*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Chunyuan Deng, Yilun Zhao, Xiangru Tang, Mark Gerstein, and Arman Cohan. 2024. Investigating data contamination in modern benchmarks for large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8706–8719, Mexico City, Mexico. Association for Computational Linguistics.

Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024. Evaluating large language models in class-level code generation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*.

Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL empowered by large language models: A benchmark evaluation. *Proceedings of the VLDB Endowment*, 17(5):1132–1145.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring coding challenge competence with APPS. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

Xiaochong Jiang, Shiqi Yang, Wenting Yang, Yichen Liu, and Cheng Ji. 2026. SOK: A taxonomy of attack vectors and defense strategies for agentic supply chain runtime. *Preprint*, arXiv:2602.19555.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems*, volume 35, pages 22199–22213.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, volume 36.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*.

Yike Zhang, Zuodong Xiang, and Hailu Xu. 2026. Performance-efficiency trade-offs in human preference prediction: A comparative study of traditional machine learning and large language models. In *Proceedings of the 31st IEEE Symposium on Computers and Communications (ISCC)*. IEEE. To appear.

Jingming Zhuo, Songyang Zhang, Xinyu Fang, Haodong Duan, Dahua Lin, and Kai Chen. 2024. ProSA: Assessing and understanding the prompt sensitivity of LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1950–1976, Miami, Florida, USA. Association for Computational Linguistics.

A Prompt Templates

A.1 Detailed Prompt

```
<role>
You are an expert Python algorithm engineer
    solving deterministic
LeetCode-style problems.
</role>

<task>
Produce a correct Python solution that fits
    the provided template and
is optimized for the problem constraints.
</task>

<instructions>
1. Read the full problem statement,
    constraints, and template before
   writing code.
2. Produce the best practical algorithm for
    the stated constraints,
   aiming for the optimal time complexity
    when possible.
3. Do not return a brute-force, quadratic,
    exponential, or placeholder
   solution when the constraints require a
    more efficient algorithm.
4. Use the required method signature from
    the template exactly.
5. Ensure the algorithm is complete and
    submission-ready, not a sketch
   or partial attempt.
6. Handle edge cases implied by the
    statement and constraints.
7. Return one complete Python solution that
    can be submitted directly.
</instructions>

<format_requirements>
Return plain Python source code only.
Start directly with the code.
Do not include explanations, markdown fences,
     or surrounding commentary.
The code must be valid, runnable Python 3
    with no syntax errors.
The code must fit the provided code template
     without requiring manual edits.
The final code should be written for
    Accepted-style performance, not just
sample-case correctness.
Only include comments when they clarify non-
    obvious logic.
</format_requirements>
```

```
<problem>
{problem_description}
</problem>

<code_template>
{code_template}
</code_template>
```

A.2 Minimal Prompt

```
<role>
You are a Python coding assistant solving a
    deterministic
LeetCode-style problem.
</role>

<task>
Write one correct and efficient Python
    solution that fits the provided
template.
</task>

<instructions>
1. Infer an algorithm that matches the
    constraints before writing code.
2. Prefer the optimal or near-optimal
    solution for the input limits.
3. Do not output brute-force code unless the
     constraints clearly make
   it acceptable.
4. Use the exact method signature from the
    template.
5. Return only final submission code.
</instructions>

<format_requirements>
Return plain Python source code only.
Start directly with the code.
Do not include explanations, markdown fences,
     or extra text.
The code must be valid, runnable Python 3
    with no syntax errors.
The code must fit the provided code template
     without requiring manual edits.
The solution must be efficient enough to
    avoid obvious Time Limit
Exceeded outcomes under the given
    constraints.
</format_requirements>

<problem>
{problem_description}
</problem>

<code_template>
{code_template}
</code_template>
```

B Model Snapshots

Checklist

The checklist follows the guidelines from the ACL Responsible NLP Research framework. For each question, the answer should be **Yes**, **No**, or **N/A**. The answer **Yes** means that the paper provides the relevant information. If the answer is **No**, an explanation must be given. If the answer is **N/A**, an explanation may be given.

1. **Did the paper include a section about limitations? Yes.** The paper contains a dedicated "Limitations" section (Section 5) that discusses benchmark size, contamination risk, API non-determinism, metric compression, o4-mini's undocumented decoding regime, prompt contrast limitations, and scope restrictions.

2. **Did the paper include a section about ethical considerations? Yes.** The paper contains a dedicated "Ethics Statement" section addressing platform-use norms, API terms of service, dual-use concerns around automated code generation, and the absence of human subjects or personal data.

3. **Did you describe the limitations of your work? Yes.** Six distinct limitations are enumerated in the Limitations section: benchmark scope ($N = 100$, one language), contamination risk, narrow experimental protocol, API opacity, metric compression (binary pass/fail), and o4-mini's uncontrolled decoding.

4. **Did you discuss any potential negative societal impacts of your work? Yes.** The Ethics Statement acknowledges that improved repeated-run reliability could lower barriers to generating code in settings lacking adequate security review, and frames the contribution as an evaluation methodology rather than an endorsement of autonomous deployment.

5. **Have you read the ethics review guidelines and ensured that your paper conforms to them? Yes.**

Artifacts

6. **Did you describe the study design, the creation of the evaluation benchmark, and the data collection process? Yes.** Section 3 ("Experimental Setup") describes problem selection by recency rank, difficulty distribution, grading criteria, model families, decoding parameters, prompt templates, normalization, and execution pipeline.

Table 3: Model snapshots used in the experiments. The API identifier column records the concrete model string used by the evaluation pipeline. Access dates are derived from the archived run dates for the corresponding completed paper runs.

Display Name	Provider	API Identifier	Access Date
GPT-4.1	OpenAI	gpt-4.1	2026-04-01
GPT-4.1-mini	OpenAI	gpt-4.1-mini	2026-03-31
o4-mini	OpenAI	o4-mini	2026-04-06
Claude Sonnet 4.5	Anthropic	claude-sonnet-4-5-20250929	2026-04-03
Claude Haiku 4.5	Anthropic	claude-haiku-4-5-20251001	2026-04-03
Gemini 3.1 Pro (preview)	Google	gemini-3.1-pro-preview	2026-03-28
Gemini 3 Flash (preview)	Google	gemini-3-flash-preview	2026-03-29
Gemini 3.1 Flash-Lite (preview)	Google	gemini-3.1-flash-lite-preview	2026-03-30
Gemini 2.5 Flash	Google	gemini-2.5-flash	2026-04-05
Gemini 2.5 Flash+Think	Google	gemini-2.5-flash (thinkingBudget=8192)	2026-04-05
QwQ-Plus	Qwen	qwq-plus	2026-03-23
Qwen-Plus	Qwen	qwen-plus	2026-03-29
Qwen-Max	Qwen	qwen-max	2026-03-23
Qwen-Turbo	Qwen	qwen-turbo	2026-03-23
DeepSeek-R1	DeepSeek	deepseek-r1	2026-03-24
DeepSeek-V3.2	DeepSeek	deepseek-v3.2	2026-03-25

7. **Did you use existing data or create new data? Yes** (existing data). The evaluation benchmark consists of the 100 most recently published LeetCode problems at collection time. No new dataset was created; the problems and test cases are sourced from the LeetCode platform.

8. **Did you discuss whether and how consent was obtained from people whose data you're using, if it applies? N/A.** The problems are algorithmically specified tasks containing no personal data. No personal identifiers or user-generated content beyond algorithmic problem statements are used.

9. **Did you discuss whether the data you are using/curating contains personally identifiable information (PII)? Yes.** Section 3 explicitly states that the benchmark "consist[s] of algorithmic specifications only and contain[s] no personally identifying information."

10. **Did you provide documentation of the artifacts, e.g., coverage of domains and languages, demographic groups represented, etc.? Yes.** Section 3 documents benchmark composition: 100 problems, three difficulty tiers (20 Easy / 50 Medium / 30 Hard), algorithmic topics, Python-only evaluation, and selection methodology. Model snapshots (provider, API identifier, access date) are listed in Appendix B.

11. **Did you report the basic statistics of the data you used? Yes.** Benchmark statistics (size, difficulty split, topic coverage) are reported in Section 3.1. Per-model result distributions appear in Table 1, Figure 2, and Figure 3.

Computational experiments

12. **If you ran experiments, did you report the number of parameters for each model? No.** Parameter counts for the evaluated models are not publicly disclosed by all provider families (OpenAI, Anthropic, Google, Qwen, DeepSeek). The paper reports the API identifiers, model tier designations, and access dates in Table 1 and Appendix B as the available model descriptors.

13. **If you ran experiments, did you report the hyperparameters used and the range of hyperparameters you searched over? Yes.** Section 3.3 specifies temperature $T = 0.3$, top-$p = 0.9$, $R = 5$ runs per problem, and the default 4,096-token output budget. Reasoning model budget exceptions are documented. The study does not perform hyperparameter search; values are fixed by experimental design.

14. **Did you use the same evaluation procedure for all experiments? Yes.** All 32 model/prompt configurations are evaluated on

the identical 100-problem set using the same deterministic judge, normalization pipeline, and metric definitions (RLPR, PSR, AV).

15. **Did you report the total amount of compute used to run your experiments? No.** API-based inference costs depend on provider pricing and token consumption, which vary per model and are not uniformly available. The paper reports the number of evaluation instances (16,000 total) and the per-model output token budgets as proxies for compute scale. Exact compute cost in GPU-hours or dollar amounts is not reported because all inference is accessed through commercial APIs without access to underlying hardware metrics.

16. **Did you use human annotators (e.g., crowdworkers) or research participants in any way? No.** All evaluation is automated. The judge is a deterministic code execution harness; no human annotation is used.

17. **Did you report the instructions given to annotators and their payment/compensation? N/A.** No human annotators were used.

18. **Did you discuss potential biases in the human annotation process and/or possible mitigation strategies? N/A.** No human annotation was used.

Model development (if applicable)

19. **Did you report model validation results? N/A.** The paper does not train or fine-tune models; it evaluates pre-existing commercial models through provider APIs.

20. **Did you report the results of any ablation study? Yes.** A controlled prompt ablation (DETAILED vs. MINIMAL) is reported in Section 4.5 with paired McNemar tests. A reasoning-vs.-standard architecture comparison is reported in Section 4.4. Both analyses are explicitly scoped as exploratory given sample-size constraints.

21. **Did you use the same evaluation procedure for all model variants? Yes.** Identical metrics, dataset, judge, normalization, and statistical inference procedures are applied to all 32 model/prompt configurations (with the documented exception that o4-mini does not expose temperature/top-p controls).

Computational social science / Information extraction

22. **If your work is on applications in the social sciences or other fields, did you discuss domain expertise? N/A.** The paper is an empirical NLP/ML evaluation study, not a social science or information-extraction application.

23. **Did you describe potential applications and use cases of your work in a responsible manner? Yes.** The Conclusion frames the metrics as evaluation tools for reproducibility-sensitive software engineering workflows. The Ethics Statement explicitly cautions against using improved coding performance as justification for fully autonomous code deployment without security review.